

Coding & Solution

Date	07 July 2026
Team ID	xxxxxx
Project Name	EduGenie – AI Learning Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Joshitha218/EduGenie-Google-Gemini-Powered-Learning-Assistant
Programming Language(s)	Python (backend), TypeScript / JavaScript (frontend)
Framework(s) Used	Framework FastAPI Frontend HTML5, CSS3, JavaScript, Jinja2 Database SQLite AI Models Google Gemini 1.5 Pro LaMini-Flan-T5

Key Features Implemented	● User Authentication (JWT) ● Student Dashboard ● Google Gemini Q&A Assistant ● Concept Explainer ● AI Quiz Generator ● Educational Content Summarizer ● Personalized Learning Roadmap ● Study History ● User Profile ● Analytics Dashboard ● Responsive UI ● REST APIs using FastAPI																						
Pending Features	No major pending features.																						
Setup Instructions	Clone the repository. Install dependencies using requirements.txt. Configure the .env file with your Gemini API Key. Start the FastAPI server: uvicorn backend.app.main:app --reload Open http://127.0.0.1:8000.																						
Code Quality Checklist	<table><thead><tr><th>Criteria</th><th>Status</th></tr></thead><tbody><tr><td>Modular Code</td><td>Yes</td></tr><tr><td>Meaningful Variable Names</td><td>Yes</td></tr><tr><td>Proper Function Naming</td><td>Yes</td></tr><tr><td>Comments & Documentation</td><td>Yes</td></tr><tr><td>Error Handling</td><td>Yes</td></tr><tr><td>REST API Architecture</td><td>Yes</td></tr><tr><td>Authentication</td><td>Yes</td></tr><tr><td>Database Integration</td><td>Yes</td></tr><tr><td>Gemini API Integration</td><td>Yes</td></tr><tr><td>Application Runs Successfully</td><td>Yes</td></tr></tbody></table>	Criteria	Status	Modular Code	Yes	Meaningful Variable Names	Yes	Proper Function Naming	Yes	Comments & Documentation	Yes	Error Handling	Yes	REST API Architecture	Yes	Authentication	Yes	Database Integration	Yes	Gemini API Integration	Yes	Application Runs Successfully	Yes
Criteria	Status																						
Modular Code	Yes																						
Meaningful Variable Names	Yes																						
Proper Function Naming	Yes																						
Comments & Documentation	Yes																						
Error Handling	Yes																						
REST API Architecture	Yes																						
Authentication	Yes																						
Database Integration	Yes																						
Gemini API Integration	Yes																						
Application Runs Successfully	Yes																						
Code Layout, Readability & Reusability Parameter Consistent Indentation Proper File Structure Meaningful Variable	Code Quality Checklist Standard Python formatting followed Modular FastAPI architecture																						

Names	Self-explanatory variable names			
Function Naming	Descriptive API methods			
Comments	Important sections documented			
Modular Design	Independent reusable modules			
No Redundant Code	Optimized implementation			
Error Handling	Exception handling implemented			
Reusable Components	Component	Technology	Reusability	
	Authentication Module	FastAPI + JWT	High	
	Gemini Service	Gemini API	High	
	Explanation Module	LaMini-Flan-T5	High	
	Quiz Module	Gemini API	High	
	Summary Module	Gemini API	High	
	Roadmap Module	Gemini API	High	
	Database Module	SQLite	High	
	Dashboard Module	FastAPI	Medium	
Overall Code Quality	Aspect	Rating		
	Code Layout	★★★★★★		
	Readability	★★★★★★		
	Reusability	★★★★★★		
	Documentation	★★★★★★		
	Overall Score	★★★★★★		
Functional Features Included	Feature	Module	Status	
	User Authentication	Authentication	Done	
	Dashboard	Dashboard	Done	
	Q&A Assistant	Gemini AI	Done	
	Concept Explainer	LaMini-Flan-T5	Done	
	Quiz Generator	Gemini AI	Done	
	Content Summarizer	Gemini AI	Done	
	Learning Roadmap	Gemini AI	Done	
	Study History	Database	Done	
	User Profile	Profile Module	Done	
REST APIs	FastAPI	Done		
Feature Summary	Metric	Value		
	Total Features Planned	10		
	Total Features Implemented	10		
	Core Features	6		
	Additional Features	4		
	Features Tested	All Features Successfully Verified		

Feature Category Breakdown	Category	Count
	User Interface	5
	Backend Logic	5
	Database	1
	API Integration	2
		1
	Authentication	
		5
	AI Modules	

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

This checklist was completed via static code review of the submitted repository (backend Flask API and React/TypeScript frontend). Backend modules are split by responsibility (app.py for routing, services/preprocessing.py, services/predictor.py, services/trainer.py); frontend code is organized into components, pages, hooks, api, and utils folders. Docstrings and inline comments are present in the backend services. Try/except blocks and input validation guard the prediction, analytics, and model-info endpoints.

